

基于里程计的轮子转速调整

一、任务描述

自动驾驶汽车能够在给定地图上找到合适和路径规划算法之外，能不能精准到达目的坐标点，还需要车轮的辅助。车轮控制的精准度，决定了自动驾驶汽车能否安全快速驾驶到终点。本次实训内容同学们需要了解里程计的作用和原理，操作自动驾驶汽车，获取里程计信息，并将其信息可视化显示。

二、任务目标

- 了解编码器的工作原理。
- 掌握编码器采集程序的实现。
- 了解PID控制的原理和功能。
- 了解里程计在自动驾驶汽车中的作用。
- 掌握获取里程计信息的流程和方法，并能够通过图形化界面观察里程计数据信息。

三、任务环境

- 操作系统：Ubuntu20.04 ROS-neotic
- 工具软件：NoMachine
- 必备硬件：自动驾驶汽车

四、任务原理

4.1 编码器使用

4.1.1 编码器简介

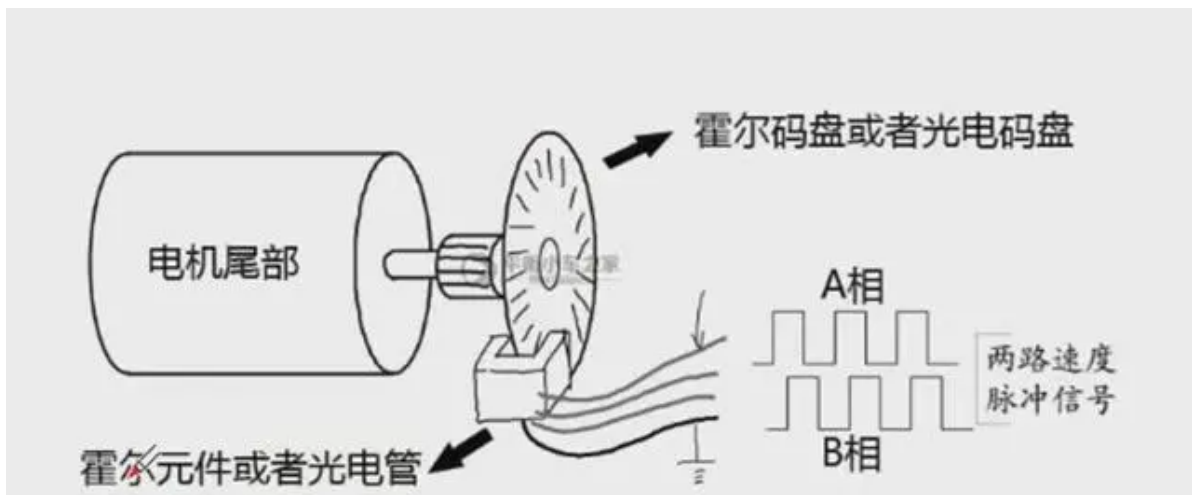
编码器是一种将角位移或者直线位移转换成一连串电数字脉冲的一种传感器。我们可以通过编码器测量电机转动的位移或者速度信息。

编码器按照工作原理，可以分为增量式编码器和绝对式编码器，绝对式编码器的每一个位置对应一个确定的数字码，因此它的示值只与测量的起始和终止位置有关，而与测量的中间过程无关。我们常用的编码器为增量式编码器。

从编码器检测原理上来分，还可以分为光学式、磁式、感应式、电容式。常见的是光电编码器（光学式）和霍尔编码器（磁式）。一般来说光电编码器是霍尔编码器精度的几十倍。

4.1.2 编码器的工作原理

光电编码器是由光码盘和光电检测装置组成。光码盘是在一定直径的圆板上等分地开通若干个长方形孔。由于光电码盘与电动机同轴，电动机旋转时，检测装置检测输出若干脉冲信号，为判断转向，一般输出两组存在一定相位差的方波信号。

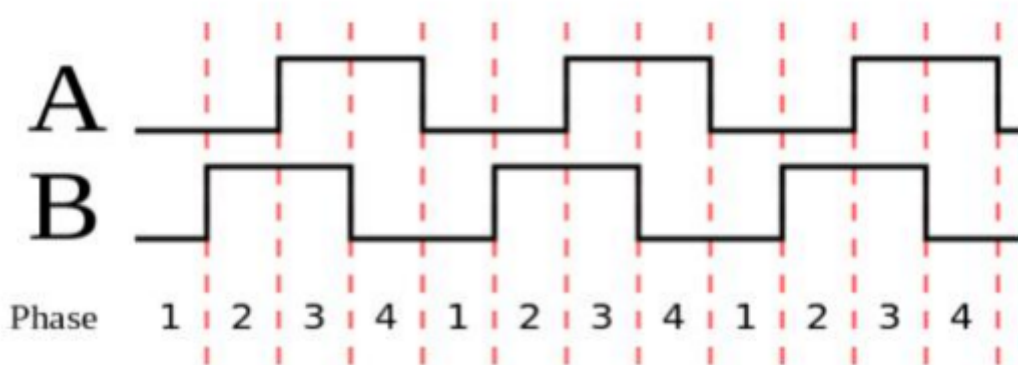


霍尔编码器是由霍尔码盘和霍尔元件组成。霍尔码盘是在一定直径的圆板上等分地布置有不同的磁极。霍尔码盘与电动机同轴，电动机旋转时，霍尔元件检测输出若干脉冲信号，为判断转向，一般输出两组存在一定相位差的方波信号。

然后我们就可以通过单片机采集 AB 相的脉冲数据，根据脉冲换算即可得到电机位置数据。

4.1.3 四倍频

四倍频是通过程序提升我们的编码器的精度的一种数据处理方法，可以有效的最大化我们的编码器的精度和测量精度。



上图是一个我们编码器输出的波形图，正常我们一般的处理方式是通过 A 相去计数，B 相去判断目前的转动方向。具体实现比如：A 相的上升沿计数或者下降沿计数，同时在 A 相的上升沿或者下降沿来根据 B 相此时的电平状态来判断转向。

四倍频则是同时计算 AB 两相的每个跳边沿，这样子原本在 A 相计数的一个脉冲周期内就实现了四次计数，从而实现了精度的提升。

4.1.4 编码器采集程序实现

因为编码器输出的是标准的正交方波，所以我们可以使用单片机（STM32 STM8 51等）直接读取。在软件中的处理方法是分两种，自带编码器接口的单片机如 STM32，可以直接使用编码器硬件接口（定时器 CH1&CH2）计数，然后从对应的寄存器读出相应的数据即可。没有编码器硬件接口的单片机，可以通过外部中断接口测量，比如把编码器 A 相输出接到单片机的外部中断接口，这样就可以通过跳边沿触发中断，然后在对应的中断服务函数里面，通过 B 相的电平来确定正反转。如当 A 相来一个跳变沿的时候，如果 B 相是高电平就认为是正转，低电平就认为是反转。

STM32 的程序实际就是初始化定时器为编码器模式即可，这里就不贴出对应的程序了，可以直接看一下我们的例程即可，下面为 Arduino 的编码器程序，更符合我们上述的逻辑，代码如下：

```

void READ_ENCODER_L() {
    if (digitalRead(ENCODER_L) == LOW) { //下降沿
        if (digitalRead(DIRECTION_L) == LOW) velocity_L--; //根据 B 判断方向 CCW
        else velocity_L++; //根据 B 判断方向 CW
    }
    else { //上升沿
        if (digitalRead(DIRECTION_L) == LOW) velocity_L++; //根据 B 判断方向 CW
        else velocity_L--; //根据 B 判断方向 CCW
    }
}
}

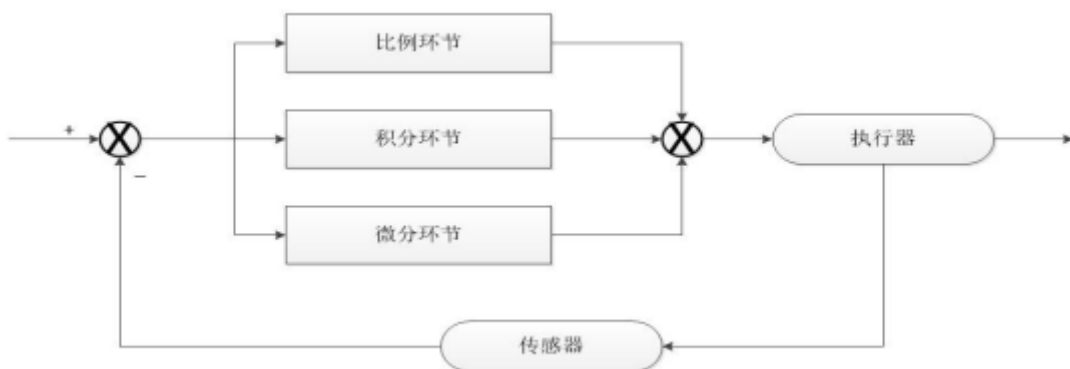
```

述代码仅在 A 相的跳边沿进行中断判断，因此仅具有二倍频的功能，我们在通过外部中断引脚来采集编码器的数据的时候也应结合资源的使用情况来综合处理。

述代码仅在 A 相的跳边沿进行中断判断，因此仅具有二倍频的功能，我们在通过外部中断引脚来采集编码器的数据的时候也应结合资源的使用情况来综合处理。

4.2 离散化与PID基础

4.2.1 PID简介



所谓 PID 控制，就是对系统偏差进行比例、积分以及微分的控制。PID 是闭环 控制，因此需要有传感器测量我们需要控制的参数，并且反馈到我们的控制计算 当中，并且参与控制。PID 由 3 个单元组成，分别是比例（Proportion）单元、积分（Integral）单元、微分（Differential）单元。通过对这三个单元的处理计算输出给执行器，达到 减小偏差最终实现收敛的过程。

示例说明：



司机大哥开车回家的一个例子：

目标值：回到家→即为距离家距离为 0m

测量值：当前离家的距离，为 10m

偏差: =目标值-测量值 (0-10) m

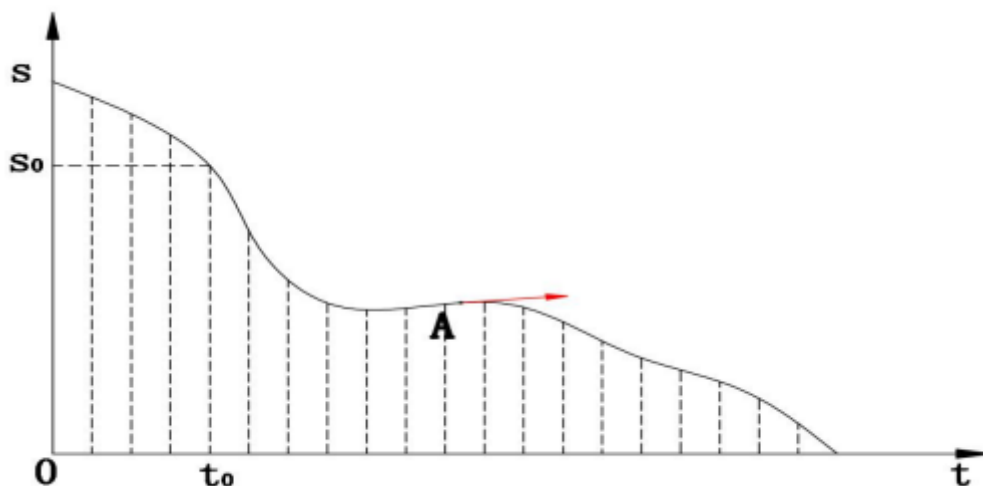
比例: 就是直接给偏差乘以一个比例系数

积分: 就是在一段时间内偏差的累积

微分: 偏差的变化率, 反馈当前的变化速率的控制

4.2.2 数据离散化

1. 传感器无法做到在时间上的连续采集
2. 微控制器 (单片机) 只需要满足系统控制需求即可实现不影响



由于我们无法做到以上的两点, 所以我们在微处理器 (单片机) 当中需要对数据进行离散化处理, 才能达到我们控制的需求。

所谓的数据离散化, 就是把无限个的数据映射到有限的空间内。在这里理解就是我们把在时间上为连续的数据 (比如速度), 通过一定的采样频率采集, 用以代表我们在连续维度上的数据。

数据离散化有两个基本的操作处理即为:

- 1) 用一阶差分代替一阶微分
- 2) 用累加代替积分

如何理解呢, 比如上图中的 A 点, 在时间连续的维度上, A 点应该对应有一个切线方向, 切线函数可以通过 $s=f(t)$ 的曲线函数求导获得。经过我们离散化处理之后我们假定 A 点前后时间点 t_1, t_2 有对应 s_1, s_2 。那么离散化之后我们认为在 $t_1 \sim t_2$ 的时间他的变化率为 $(s_2-s_1)/(t_2-t_1)$; 很显然 $\Delta t=t_2-t_1$ 应该是足够小, 小到不影响我们对系统的要求。

另外用累加代替积分这个就更容易理解了, 这个是我们在学习微积分的基础推导过程中就需要涉及到的知识。假设要求曲线与 $t=0, t=t_0, s=0$ 所围成的面积, 我们用微积分的方法很容易得知 S 面积 = $\int_0^t f(t) dt$ 。我们离散化之后则假设有 $0 \sim t_0$ 之间有四个采样周期, 采样值分别对应 s_0, s_1, s_2, s_3 ; S 面积 = $(s_0+s_1+s_2+s_3) * \Delta t$, 这样我们的单片机就可以进行数据采集和处理了。

结合到我们电机编码器的数据采集当中实际应用就是:

电机位置: 实际就是一直累积的脉冲数 (单位为脉冲)

电机速度: 实际就是对应上述中的 A 点的切线, 即测量方法为相邻的两次位置差与单位时间的比值 (单位为脉冲数/单位时间)

五、任务步骤

步骤一 创建启动脚本

创建脚本"/home/mo/ros_learning_demos/sample_shell/chapter_3_robot_driver.sh"，添加如下内容，来控制获取里程计信息的整个流程入口：

```
#!/bin/bash
#source /opt/ros/melodic/setup.bash
#source ~/ros_space/devel/setup.bash

gnome-terminal "启动底盘" -- bash -c " roslaunch learning_sensor
robot_base.launch;killall rqt"
gnome-terminal "启动rqt" -- bash -c " rqt;killall roslaunch"
```

此处启动"robot_base.launch"节点和rqt节点。

步骤二 主要流程实现

创建文

件"/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/launch/robot_base.launch"，添加如下内容，实现整个流程：

```
<launch>

  <!--ROS底盘坐标发布节点-->
  <node pkg="robot_driver" type="robot_odom_publisher"
name="robot_odom_publisher" output="screen">
    <param name="publish_tf" value = "true"/>
  </node>

  <!--ROS底盘控制节点-->
  <node pkg="robot_driver" type="robot_driver" name="base_driver"
output="screen" respawn="true" >
    <param name="sleep_mode" value = "false"/>
  </node>

  <!--机器人模型发布-->
  <include file="$(find urdf_car)launch/unionws_test.launch"/>

  <!-- 可视化界面 -->
  <node pkg="rviz" type="rviz" name="rviz" args="-d $(find
learning_sensor)/rviz/depth.rviz" output="screen" required="true"/>
</launch>
```

流程中实现了里程计发布节点，地盘控制节点以及机器人模型发布节点和可视化界面显示。

步骤三 里程计消息转换

在步骤二中，启动底盘控制节点“base_driver”，创建base_driver节点“/home/mo/ros_space/src/ai_node_master/launch/base_driver.launch”，添加如下内容，实现控制底盘运动的功能：

```
<launch>
  <!--param参数配置-->

  <!-- <param name="node_list_path" value="/ros_space/src/ai_node_master/launch"
  /> -->
  <!-- <node pkg="ai_node_master" type="ros_starter" name="node_starter"
  output="screen" respawn="true" /> -->
  <arg name="encoder_only"          default="false" />
  <arg name="use_API"               default="false" />
  <arg name="driver_node"           default="true" />
  <rosparam file="/home/mo/ros_space/encoder.yaml" command="load" />
  <node pkg="ai_node_master" type="ros_topic_checker" name="topic_checker"
  output="screen" respawn="true" />

  <!--ROS底盘坐标发布节点-->
  <node pkg="robot_driver" type="robot_odom_publisher" name="robot_odom_publisher"
  output="screen">
    <param name="publish_tf" value = "${arg encoder_only}"/>
  </node>

  <!--ROS底盘控制节点-->
  <node if="${arg driver_node}" pkg="robot_driver" type="robot_driver"
  name="base_driver" output="screen" respawn="true" >
    <param name="sleep_mode" value = "false"/>
  </node>

  <!--位置校准节点-->
  <group unless="$(eval encoder_only )">
    <node pkg="laser_scan_matcher" type="laser_scan_matcher_node"
    name="laser_scan_matcher_node">

      <param name="fixed_frame" value = "odom"/>
      <param name="base_frame" value = "base_footprint"/>
      <param name="use_imu" value = "false"/>
      <param name="use_odom" value = "true"/>
      <param name="kf_dist_linear" value = "0.03"/>
      <param name="kf_dist_angular" value = "0.03"/>
      <param name="outliers_maxPerc" value = "0.85"/>
      <!-- <param name="max_angular_correction_deg" value = "0.9"/> -->
      <param name="use_point_to_line_distance" value = "1"/>

      <param name="max_iterations" value="10"/>
    </node>
  </group>

  <!--机器人模型发布-->
  <include file="$(find urdf_car)launch/unionws_test.launch"/>

</launch>
```

此处启动底盘坐标发布节点“robot_odom_publisher”和底盘控制节点“robot_driver”，创建“/home/mo/ros_space/src/robot_driver/src/robot_odom_publisher.cpp”，添加如下内容，实现里程计消息的发布：

```
#include <ros/ros.h>
#include <robot_driver/car_vel.h>
#include <nav_msgs/Odometry.h>
#include <geometry_msgs/TransformStamped.h>
#include <thread>
#include <math.h>
#include <tf2_msgs/TFMessage.h>
#include <tf/transform_broadcaster.h>
using namespace std;
ros::Publisher odom_pub;
ros::Publisher tf_pub;
float posdata[3];
double last_time;
bool publish_tf;

void odomCallback(const robot_driver::car_vel &encoder)
{
    // TF与ODOM参数
    nav_msgs::Odometry odomdata;
    tf::TransformBroadcaster tfbc;
    // tf::Transform transform;
    // 计算时间间隔dt
    double now_time = ros::Time().now().toSec();
    double dt = now_time - last_time;
    last_time = now_time;

    // 位置x, y与旋转值w赋值
    posdata[2] += encoder.angle;
    posdata[0] += encoder.throttle * cos(posdata[2]) - encoder.rampage *
    sin(posdata[2]);
    posdata[1] += encoder.throttle * sin(posdata[2]) + encoder.rampage *
    cos(posdata[2]);

    // 计算四元数
    tf::Quaternion q;
    q.setRPY(0,0,posdata[2]);

    // odom参数赋值
    odomdata.child_frame_id = "base_footprint";
    odomdata.header.frame_id = "odom";

    odomdata.header.stamp = ros::Time().now();

    odomdata.pose.pose.position.x = posdata[0];
    odomdata.pose.pose.position.y = posdata[1];
    odomdata.pose.pose.position.z = 0;
    odomdata.pose.pose.orientation.x = q[0];
    odomdata.pose.pose.orientation.y = q[1];
    odomdata.pose.pose.orientation.z = q[2];
    odomdata.pose.pose.orientation.w = q[3];

    if (publish_tf == true){
        // TF赋值
```

```

// ROS_INFO("%i",int(publish_tf));
geometry_msgs::TransformStamped tfSt;
tf2_msgs::TFMessage tfmsg;
tfSt.child_frame_id = "base_footprint";
tfSt.header.frame_id = "odom";

tfSt.header.stamp = ros::Time().now();

tfSt.transform.translation.x = posdata[0];
tfSt.transform.translation.y = posdata[1];
tfSt.transform.translation.z = 0;
tfSt.transform.rotation.x = q[0];
tfSt.transform.rotation.y = q[1];
tfSt.transform.rotation.z = q[2];
tfSt.transform.rotation.w = q[3];

tfmsg.transforms.push_back(tfSt);
tf_pub.publish(tfmsg);
}

// transform.setOrigin(tf::Vector3(posdata[0],posdata[1],0));
// transform.setRotation(q);
// 发布消息
// tfbc.sendTransform( tf::StampedTransform( tf::Transform(tf::Quaternion(0, 0,
0, 1), tf::Vector3(0.1, 0.0, 0.2)), ros::Time::now(), "map",
"base_footprint"));
// tfbc.sendTransform(tf::StampedTransform(transform, ros::Time().now(), "odom",
"base_footprint"));
odom_pub.publish(odomdata);

// ROS_INFO("%d",now_time);
sleep(0.05);
}

int main(int argc, char** argv)
{
    ros::init(argc, argv, "odom_publisher");
    ros::NodeHandle nh;
    ros::param::param<bool>("~publish_tf", publish_tf, true);
    // ROS_INFO("-----%i",int(publish_tf));
    odom_pub = nh.advertise<nav_msgs::Odometry>("odom", 5);
    tf_pub = nh.advertise<tf2_msgs::TFMessage>("tf", 5);
    ros::Subscriber encoder = nh.subscribe("encoder",5,odomCallback);
    ros::Rate rate(20);
    rate.sleep();
    ros::spin();
}

```

创建“/home/mo/ros_space/src/robot_driver/src/robot_driver.cpp”文件，添加如下内容，实现底盘的控制：

```

#include "robot_driver/robot_driver.h"
#include <robot_driver/car_vel.h>
#include <thread>
#include <math.h>
#include <tf/tf.h>

```



```

#define PI acos(-1)
using namespace std;

serial::Serial carSerial;
float serialCmd[5];
double last_time;
float prm, k_x, k_y, k_r ,car_width, car_lenght, pub_rate;
float yaw , x_max, y_max, w_max;
int battery_count = 1;
float battery = 0;
bool sleep_mode;

ros::Publisher imu_pub;
ros::Publisher encoder_pub;

/* 串口初始化 */
bool serialInit()
{
    carSerial.setPort("/dev/car_base");
    carSerial.setBaudrate(115200);
    serial::Timeout t = serial::Timeout::simpleTimeout(1000);
    carSerial.setTimeout(t);
    for (int i = 0; i<3; i++)
    {
        try
        {
            carSerial.open();
            if (carSerial.isOpen())
                cout<<"open serial"<<endl;
            return true;
        }
        catch(serial::IOException& e)
        {
            cout << "trying " << i <<endl;
            sleep(2);
        }
    }
    cout<<"cannot open serial"<<endl;
    return false;
}

/* 多线程串口读取，如果格式正确，则执行msgPublisher发布话题，
如果不正确，则直接输出 */
void serialRead()
{
    ros::Rate r2(20);
    while (ros::ok())
    {
        // 在队列中的字符串个数，这个比较重要，关乎读取数据的准确性
        size_t n = carSerial.available();
        if (n>24)
        {
            string str_msg= "";
            size_t i = carSerial.read(str_msg,n);

            //如果格式正确

```

```

        int data[9] = {0};
        for(int k = 0; k < int(n/24); k++)
        {
            if (str_msg[k*24] == char(0xff) && str_msg[k*24+1] == char(0xd8)
&& str_msg[k*24+21] == char(0xd9) && str_msg[k*24+22] == char(0x00))
            {
                // 高低位合并，并赋值给传感器数据data[]数组

                for (int i = 0; i < 9; i++)
                {
                    int16_t temp = uint8_t(str_msg[k*24+2*i+2]) +
(uint8_t(str_msg[k*24+2*i+3])<<8);
                    data[i] += temp;
                }
                // 发布话题

                if (battery_count>int(100))
                {
                    serialCmd[4] = battery;
                    battery_count = 1;
                    battery = int(str_msg[20]);
                }else if(battery == 0){
                    battery = int(str_msg[20]);
                    serialCmd[4] = battery;
                }else{
                    battery = (battery*battery_count+
int(str_msg[20]))/(battery_count + 1);
                    battery_count = battery_count+1;
                }

            }
        }
        msgPublisher(data);
    }
    r2.sleep();
}

}

/* 监听速度话题的回调方法，将serialCmd赋值，然后执行serialwrite函数写入串口 */
void velCallback(const geometry_msgs::Twist &data)
{
    serialCmd[0] = inside(-w_max, w_max,data.angular.z);
    serialCmd[1] = inside(-x_max, x_max,data.linear.x);
    serialCmd[2] = inside(-y_max, y_max,data.linear.y);
    if (!sleep_mode)
        {serialwrite(serialCmd);}
}

float inside (float min, float max , float x)
{
    if (max < x)
    {
        return max;
    }

    if (min > x)
    {
        return min;
    }
}

```

```

    }

    return x;
}

/* 串口写入函数，使用将float的数组映射为1~254的整数，再写入给串口设备
参数是测试出来的 */
void serialWrite(float sData[])
{
    // 使用将float的数组映射为1~254的整数
    string serialData = "";
    char angle = round(mapTo(sData[0], -w_max, w_max, 1, 254));
    char throttle = round(mapTo(sData[1], x_max, -x_max, 1, 254));
    char rampage = round(mapTo(sData[2], y_max, -y_max, 1, 254));
    char pitch = round(mapTo(sData[3], -1, 1, 1, 254));
    char shot = int(sData[4]);

    // 给字符串赋值
    serialData.push_back(0xff);
    serialData.push_back(0xd8);
    serialData.push_back(angle);
    serialData.push_back(throttle);
    serialData.push_back(rampage);
    serialData.push_back(pitch);
    serialData.push_back(shot);
    serialData.push_back(0x00);
    serialData.push_back(0xd9);
    serialData.push_back(0x00);

    // 向串口发送
    carSerial.write(serialData);
}

/* 话题发布函数，发布小车里里程计信息（car_vel类型）与IMU信息 */
void msgPublisher(int msgData[])
{
    double now_time = ros::Time().now().toSec();
    double dt = now_time - last_time;
    last_time = now_time;
    // ROS_INFO("%f",dt);
    // cout << ros::Time().now() << endl;
    // 如果x,y,z加速度都不等于0（地球上重力加速度为9.8），则判断IMU连接成功，执行计算并发布IMU话题
    if (msgData[3] != 0 && msgData[4] != 0 && msgData[5] != 0)
    {
        sensor_msgs::Imu imu_data;

        imu_data.header.stamp = ros::Time().now();
        imu_data.linear_acceleration.x = msgData[3];
        imu_data.linear_acceleration.y = msgData[4];
        imu_data.linear_acceleration.z = msgData[5];

        imu_data.angular_velocity.x = msgData[6];
        imu_data.angular_velocity.y = msgData[7];
        imu_data.angular_velocity.z = msgData[8];
    }
}

```

```

        // 计算RPY值
        float radius = sqrt(msgData[3]*msgData[3] + msgData[4]*msgData[4] +
msgData[5]*msgData[5]);
        float rall = asin(-float(msgData[3])/radius);
        float picth = atan(float(msgData[4])/msgData[5]);
        yaw = yaw + msgData[8] * dt / 1000;

        // 使用RPY计算四元数
        tf::Quaternion q;
        q.setRPY(rall,picth,yaw);

        imu_data.orientation.x = q[0];
        imu_data.orientation.y = q[1];
        imu_data.orientation.z = q[2];
        imu_data.orientation.w = q[3];

        // 发布IMU话题
        imu_pub.publish(imu_data);
    }

    // encoder 赋值并发布话题,
    float angle = -float(msgData[0]) / 660 * 0.049 * PI * k_r *prm/330
*0.49/(car_width + car_lenght);
    float throttle = -float( msgData[1]) / 660 * 0.049 * PI * k_x *prm/330;
    float rampage = float(msgData[2]) / 660 * 0.049 * PI * k_y *prm/330;

    robot_driver::car_vel endata;
    endata.angle = angle;
    endata.throttle = throttle;
    endata.rampage = rampage;
    encoder_pub.publish(endata);
}

/* 映射函数, 将浮点型映射为整形 */
int mapTo(float x, float in_min, float in_max, float out_min, float out_max)
{
    return round((x - in_min) * (out_max - out_min) / (in_max - in_min) +
out_min);
}

/* 主函数 */
int main(int argc, char** argv)
{
    ros::init(argc, argv, "driver");
    ros::NodeHandle nh;

    ros::param::param<float>("k_r", k_r, 1);
    ros::param::param<float>("k_x", k_x, 1);
    ros::param::param<float>("k_y", k_y, 1);
    ros::param::param<float>("prm", prm, 178);
    ros::param::param<float>("car_width", car_width, 0.25);
    ros::param::param<float>("car_lenght", car_lenght, 0.24);
    ros::param::param<bool>("~sleep_mode", sleep_mode, false);
    ros::Subscriber sub;
    if (sleep_mode)
    {pub_rate = 0.2;}
}

```

```

else
{
    pub_rate = 20;
    sub = nh.subscribe("/cmd_vel",1,velCallback);
}

x_max = 1.56*prm/330;
y_max = 1.72*prm/330;
w_max = 5.9*prm/330;

encoder_pub = nh.advertise<robot_driver::car_vel>("encoder", 10);
imu_pub = nh.advertise<sensor_msgs::Imu>("IMU", 10);

serialInit();
// serialCmd[0] = -1;
ros::Rate rate(pub_rate);
thread read(serialRead);

while (ros::ok())
{
    if (sleep_mode)
        {serialWrite(serialCmd);}
    // serialWrite(serialCmd);
    // serialRead();

    rate.sleep();
    ros::spinonce();
}
}

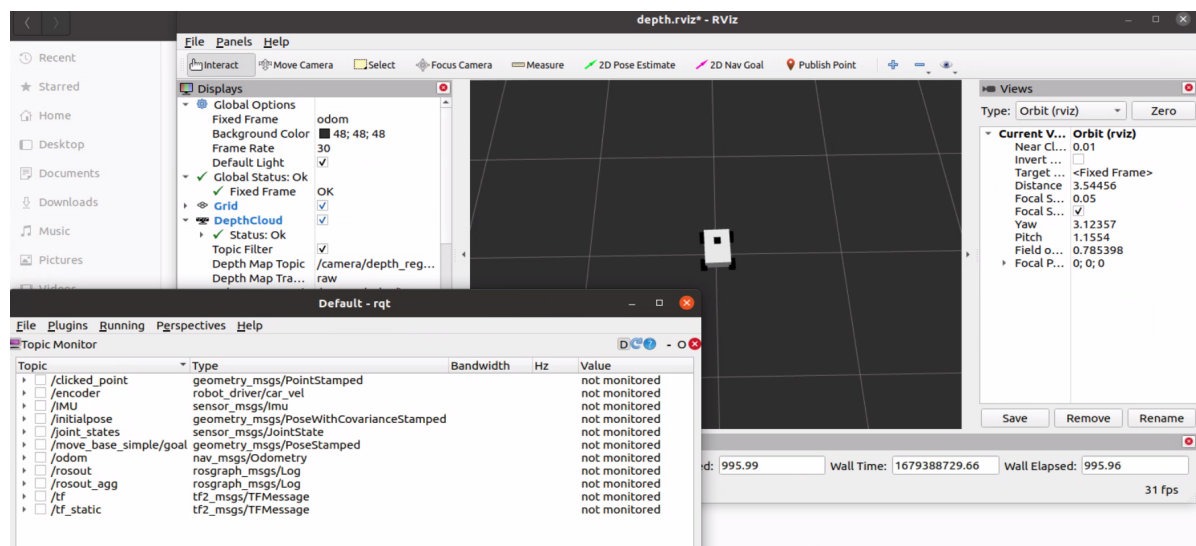
```

步骤四 实现里程计数据获取和显示

使用如下命令来执行步骤一中的启动节点“chapter_3_robot_driver.sh”:

```
sh chapter_3_robot_driver.sh
```

可以打开RVIZ和rqt, RVIZ显示小车模型, rqt显示传感器数据。



The screenshot shows the RViz interface with the following components:

- Top Bar:** Title "depth.rviz* - RViz" and standard window controls.
- Left Panel (Displays):**
 - Global Options:** Fixed Frame (odom), Background Color (48; 48; 48), Frame Rate (30), Default Light (checked).
 - Global Status:** All items (Fixed Frame, Grid, DepthCloud) are OK.
 - DepthCloud:** Status is OK, Topic Filter (checked), Depth Map Topic (/camera/depth_reg...), Depth Map Tra... (raw), Color Image Topic (/camera/color/imag...), Color Transport... (raw), Queue Size (5), Auto Size (checked).
- Top Toolbar:** Interact, Move Camera, Select, Focus Camera, Measure, 2D Pose Estimate, 2D Nav Goal, Publish Point, and navigation controls.
- Center View:** A 3D grid with a robot model (a small white square with a black square in the center) positioned at the origin.
- Right Panel (Views):**
 - Type: Orbit (rviz), Zero.
 - Current V... Orbit (rviz):**
 - Near Cl...: 0.01
 - Invert: ☐
 - Target ...: <Fixed Frame>
 - Distance: 3.54456
 - Focal S...: 0.05
 - Focal S...: ☒
 - Yaw: 3.12357
 - Pitch: 1.1554
 - Field o...: 0.785398
 - Focal P...: 0; 0; 0
- Bottom Panel (Time):**
 - Buttons: Pause, Synchronization: Off, Reset.
 - ROS Time: 1679388850.27
 - ROS Elapsed: 1116.63
 - Wall Time: 1679388850.30
 - Wall Elapsed: 1116.53

The screenshot shows the ROS Topic Monitor interface. The 'Topic' column lists various topics, and the 'Value' column displays their current values. The '/odom' topic is highlighted with a red box, indicating it is the selected topic for monitoring.

Topic	Type	Bandwidth	Hz	Value
/clicked_point	geometry_msgs/PointStamped			not monitored
/encoder	robot_driver/car_vel			not monitored
/IMU	sensor_msgs/Imu			not monitored
/initialpose	geometry_msgs/PoseWithCovarianceStamped			not monitored
/joint_states	sensor_msgs/JointState			not monitored
/move_base_simple/goal	geometry_msgs/PoseStamped			not monitored
☒ /odom	nav_msgs/Odometry	5.25KB/s	7.31	'base_footprint'
child_frame_id	string			'odom'
header	std_msgs/Header			seq: 8838
frame_id	string			
seq	uint32			
stamp	time			
pose	geometry_msgs/PoseWithCovariance			
covariance	float64[36]			(0.0, ...)
pose	geometry_msgs/Pose			
orientation	geometry_msgs/Quaternion			w: 0.9928061474885143 x: 0.0 y: 0.0 z: -0.11973284223225711
w	float64			
x	float64			
y	float64			
z	float64			
position	geometry_msgs/Point			x: -0.030489986762404442 y: 0.024772796779870987 z: 0.0
x	float64			
y	float64			
z	float64			
twist	geometry_msgs/TwistWithCovariance			
covariance	float64[36]			(0.0, ...)
twist	geometry_msgs/Twist			
angular	geometry_msgs/Vector3			x: 0.0 y: 0.0 z: 0.0
x	float64			
y	float64			
z	float64			
linear	geometry_msgs/Vector3			x: 0.0 y: 0.0 z: 0.0
x	float64			
y	float64			
z	float64			
/rosout	rosgraph_msgs/Log			not monitored
/rosout_agg	rosgraph_msgs/Log			not monitored
/tf	tf2_msgs/TFMessage			not monitored
/tf_static	tf2_msgs/TFMessage			not monitored

通过本此实训任务可以深刻理解里程计消息的发布和获取流程，编码器的工作原理和作用，为后面的课程奠定坚实基础。